

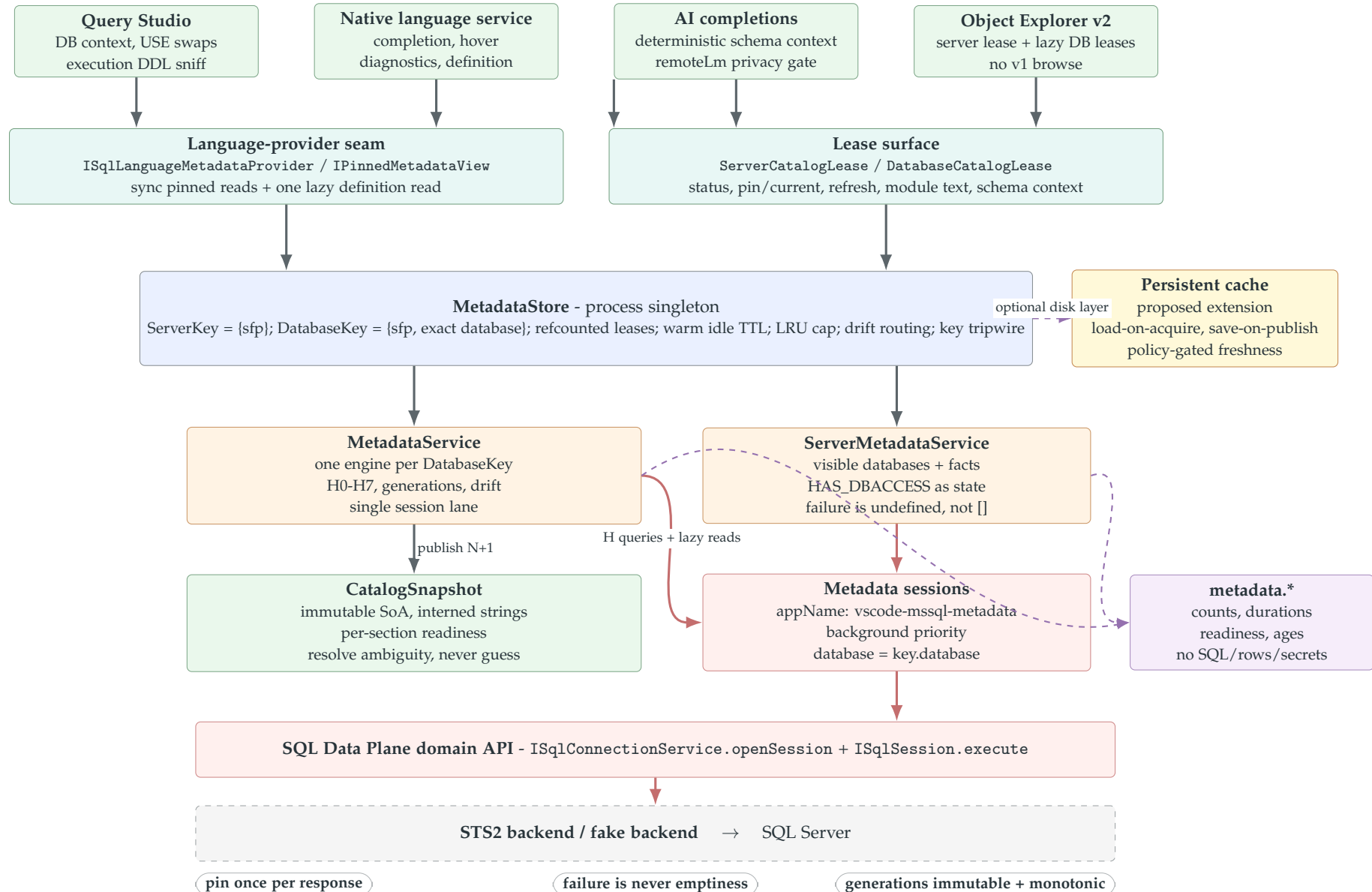
1. Metadata substrate at a glance

The substrate is strongest where it is boring: two seams above one store, immutable views below, and no consumer gets to improvise catalog truth.

Metadata substrate - review visuals v2

vscode-mssql dev/query checked 2026-07-06

store/registry pure snapshot engine/hydration sessions/data plane contract/privacy gate proposed cache observability



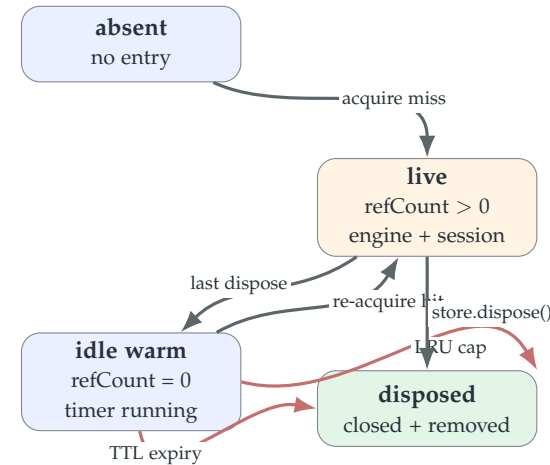
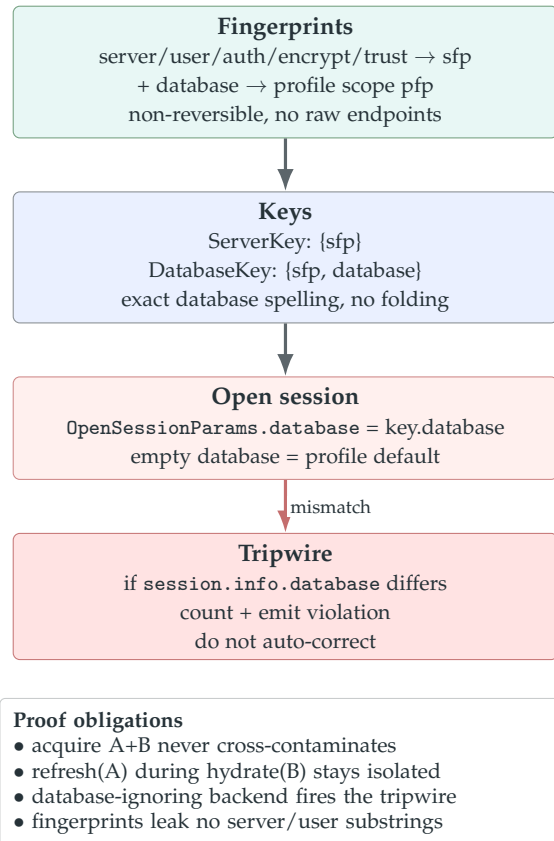
2. Identity, key-correctness, and leases

Database isolation is enforced by construction first, then watched by a tripwire. Warm memory reuse is separate from the proposed persistent cache.

Metadata substrate - review visuals v2

vscode-mssql dev/query checked 2026-07-06

store/registry pure snapshot engine/hydration sessions/data plane contract/privacy gate proposed cache observability

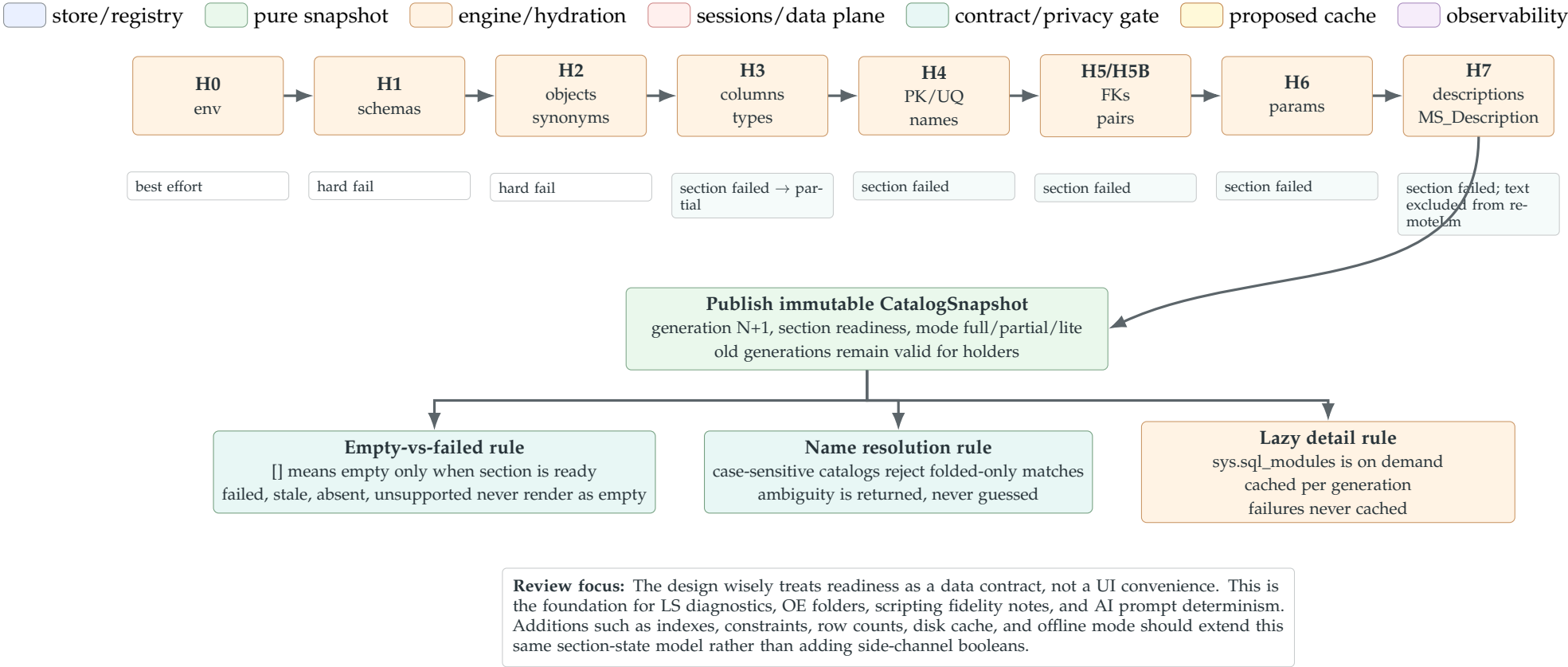


Lease rules

One entry per key. Leases are refcounted, double-dispose is inert, listeners are isolation-wrapped, and server/database generations are independent. Warm in-memory reuse keeps a session alive briefly; persistent cache, when enabled, is a separate file-backed layer and must never keep a connection alive.

3. Hydration ladder and readiness honesty

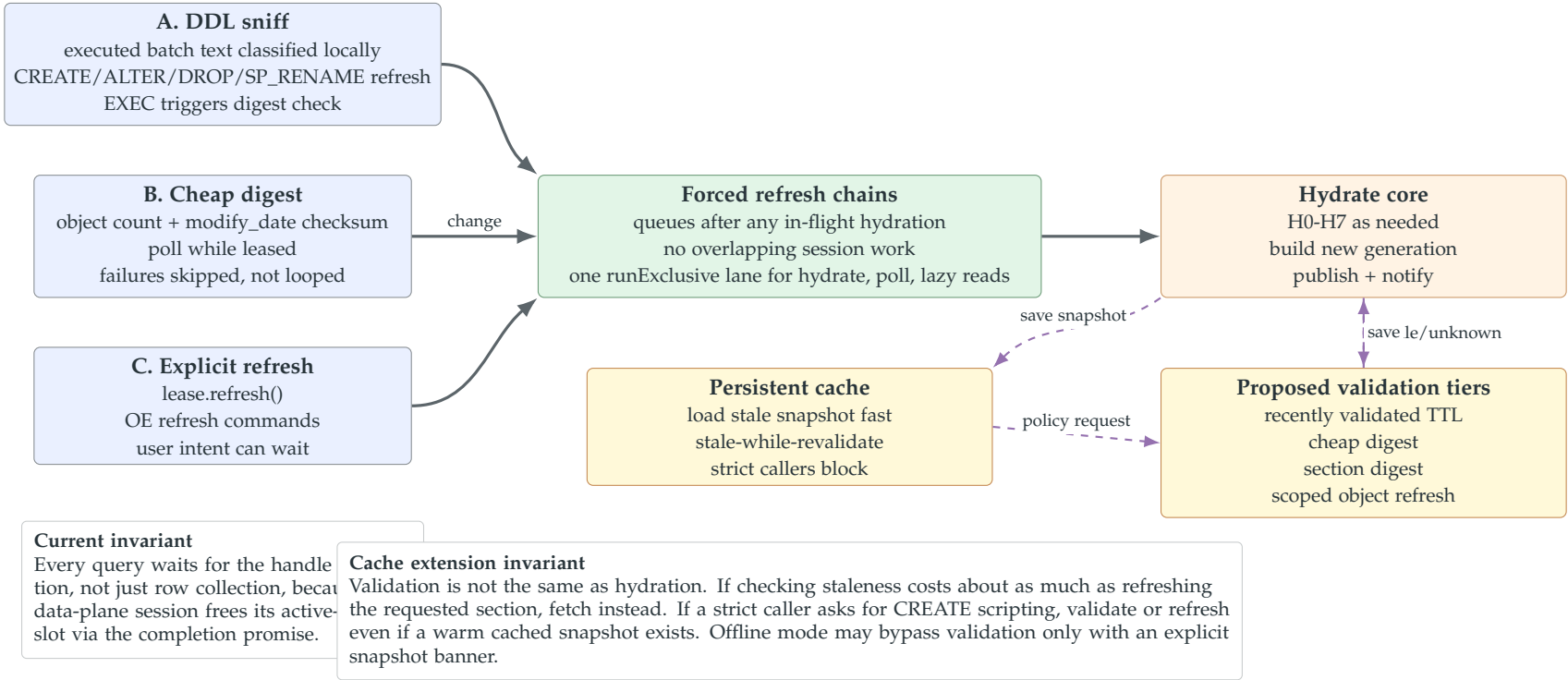
The ladder deepens in strict order, but later sections fail independently. Consumers must read section state before trusting an empty result.



4. Drift detection and refresh orchestration

The current loop is cheap and safe. The cache proposal adds policy-aware validation so strict consumers can demand fresher proof than completion needs.

store/registry pure snapshot engine/hydration sessions/data plane contract/privacy gate proposed cache observability

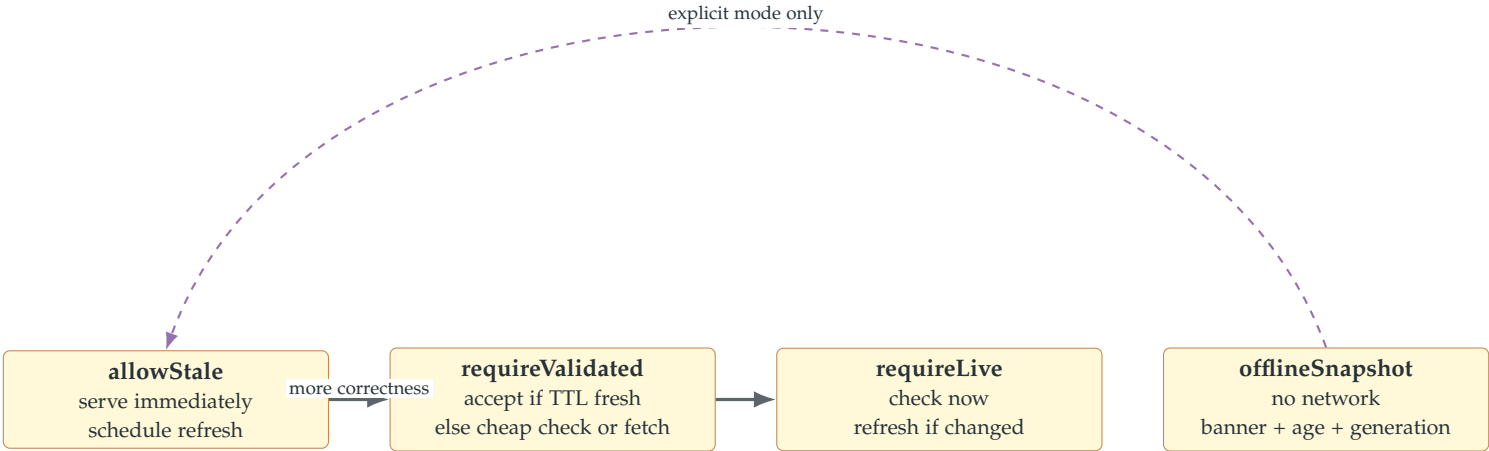


5. Consumer freshness policy matrix

One metadata store can serve fast, possibly stale suggestions and strict, live scripting only if freshness becomes an explicit request contract.

store/registry pure snapshot engine/hydration sessions/data plane contract/privacy gate proposed cache observability

Consumer	Default freshness	May use cached data	Must validate or refresh
Query Studio	recently validated, background refresh	status badges, readiness display, database dropdown during reconnect	USE-triggered lease swap and DDL notification routing
Native completions	allow stale; serve instantly	object names, columns, FK join ranking, system catalog context	none on hot path; mark incomplete when required sections are absent or failed
Native diagnostics	validated or suppress/demote	lexical and structural diagnostics; cautious hints	schema errors 207/208/209 need recently validated objects and columns or are suppressed
Hover/signature	bounded stale for local facts	types, FK badges, proc params when ready	module definitions, descriptions, and authoritative claims may request validation
AI completions	allow stale with deterministic generation marker	schema context, excluding descriptions for remoteLm	privacy gates and prompt byte-identity remain mandatory
OE v2	require recent folder validation, TTL based	folder expand if validated inside TTL; stale-while-revalidate for low-risk browse	first expand beyond TTL, refresh, dropped object path, destructive actions
Scripting/definition	require live unless offline snapshot mode	offline script with banner and fidelity notes	CREATE/ALTER script, module text, object detail, destructive command prep

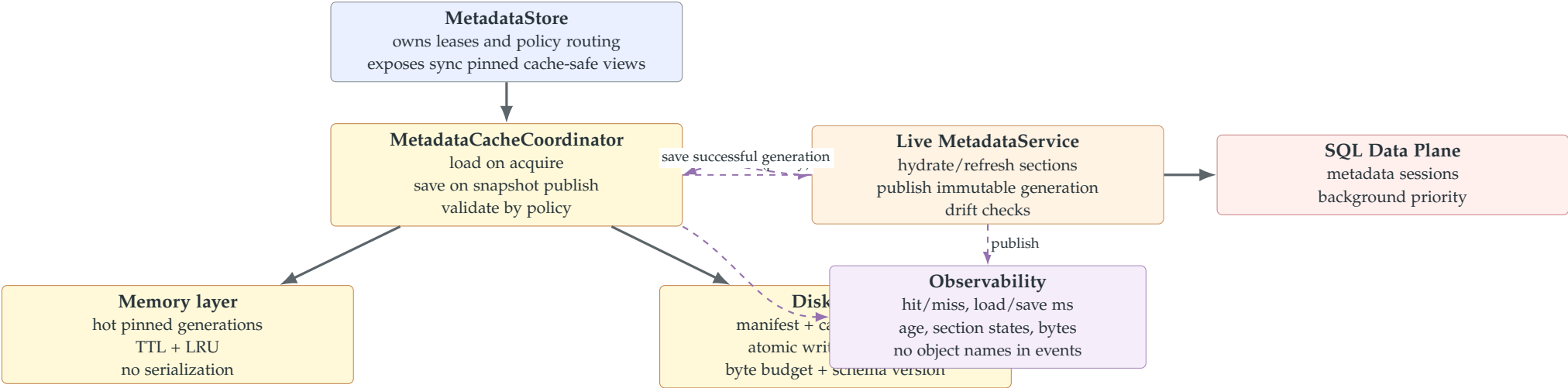


Design rule: stale data is a performance choice for suggestions, not a license for false diagnostics or wrong scripts.

6. Proposed persistent cache architecture

The disk cache is a projection of published snapshots. It loads fast, saves atomically, and never replaces live validation for strict consumers.

store/registry pure snapshot engine/hydration sessions/data plane contract/privacy gate proposed cache observability



Atomicity
write temp file, flush, rename, then manifest last. On corrupt payload or schema mismatch, delete and hydrate live.

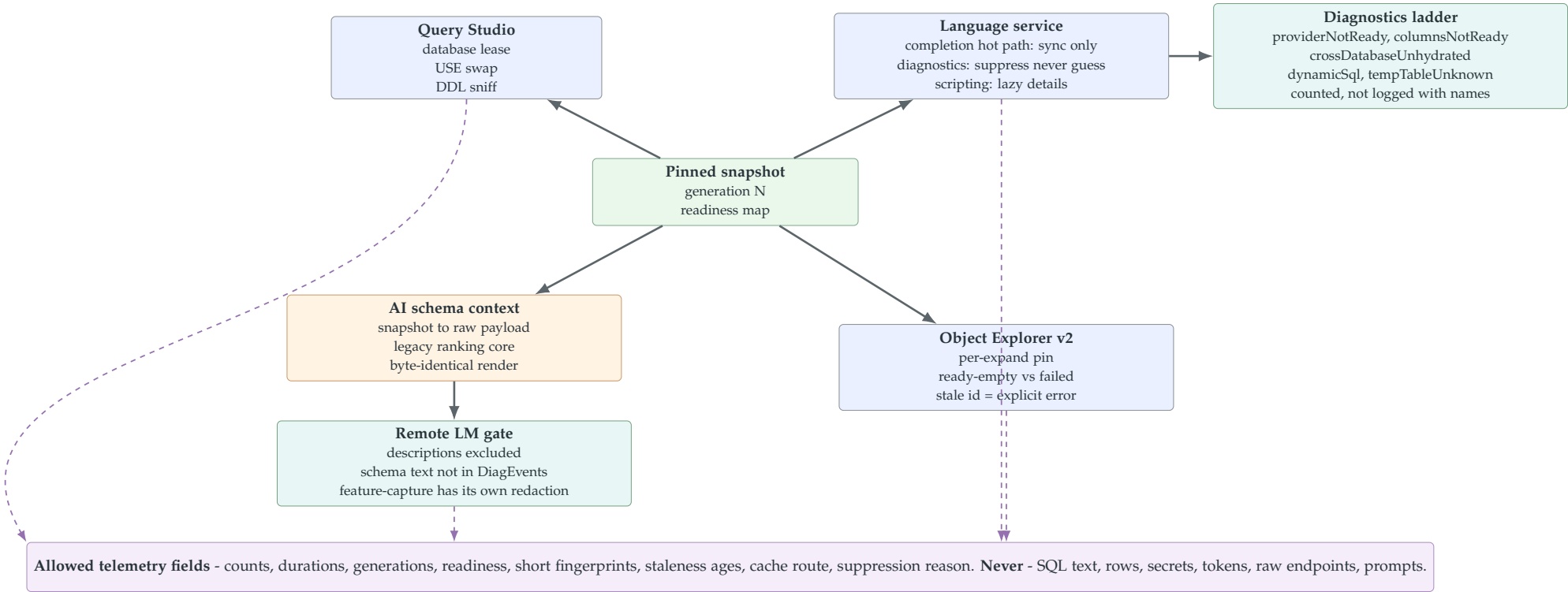
Storage shape
globalStorage/metadata-cache/v1/<serverHash>/<databaseHash>/manifest.json plus one compact payload. Start with gzip-compressed JSON using the existing SoA arrays. Keep the codec replaceable: CBOR or MessagePack can be introduced only if measured load times demand it. Never store connection strings, tokens, passwords, or remote model prompts. Module definitions and description text need explicit local-cache policy because they are user-authored content.

7. Consumer contracts and privacy gates

The same data can feed four very different surfaces only because each surface has a narrow contract and a privacy fence.

Metadata substrate - review visuals v2
vscode-mssql dev/query checked 2026-07-06

store/registry pure snapshot engine/hydration sessions/data plane contract/privacy gate proposed cache observability



8. Evidence, gaps, and next design checkpoints

The design has unusually crisp invariants. The remaining risk is mostly about persistence, strict freshness, expanded metadata coverage, and dogfood scale.

store/registry pure snapshot engine/hydration sessions/data plane contract/privacy gate proposed cache observability

Measured / built evidence	Value	Meaning
Full hydration, 10k objects / 81k columns / 2k FKs	148 ms	Unit lane through the store
listObjects() over 10,201 objects	10 ms	OE folder enumeration is not the bottleneck
Prefix search over 10k names	about 0 ms	Folded-name index is doing its job
Warm native completion over 2k-line doc	p95 0.15 ms	Hot path can stay sync over pinned views
OE v2 connect to Databases rendered	338-416 ms	Live STS2 + SQL Server scenario, 4 reps
Warm re-acquire inside TTL	instant	No re-hydration, generation unchanged

- Honest gaps to keep visible**
- persistent disk cache not built
 - indexes, default/check constraints, row counts not hydrated
 - scoped delta and section digest tiers not built
 - server-lane serialized USE remains an alternative
 - H7 descriptions truncate at 4000 chars
 - module text should not be persisted by default
 - live 10k-object dogfood server still needed

- Recommended checkpoints**
1. Add freshness policy API before disk cache.
 2. Persist only published snapshots, never raw query rows.
 3. Add cache privacy settings before storing descriptions/module text.
 4. Add drift tests that prove safe stale data and strict blocking.
 5. Make cache load/save part of perf gates before defaulting on.

